

Exemple d'affichage complet

Objectifs

- Synthétiser les éléments clés d'un chapitre.
- Illustrer avec un extrait de code concis.
- Donner 1 à 2 repères visuels (tableau, schéma).

Code Python (basique)

```
root@python:~$  
  
def somme(liste):  
    total = 0  
    for x in liste:  
        total += x  
    return total  
  
print(somme([1, 2, 3])) # 6
```

Tableau compact

Structure	Mutable	Clés/Index
list	Oui	Index entiers
tuple	Non	Index entiers
dict	Oui	Clés hachables

Note

Astuce : préférez plusieurs petites sections à une seule très dense.

Voir aussi : [Documentation Python](#).

Définitions

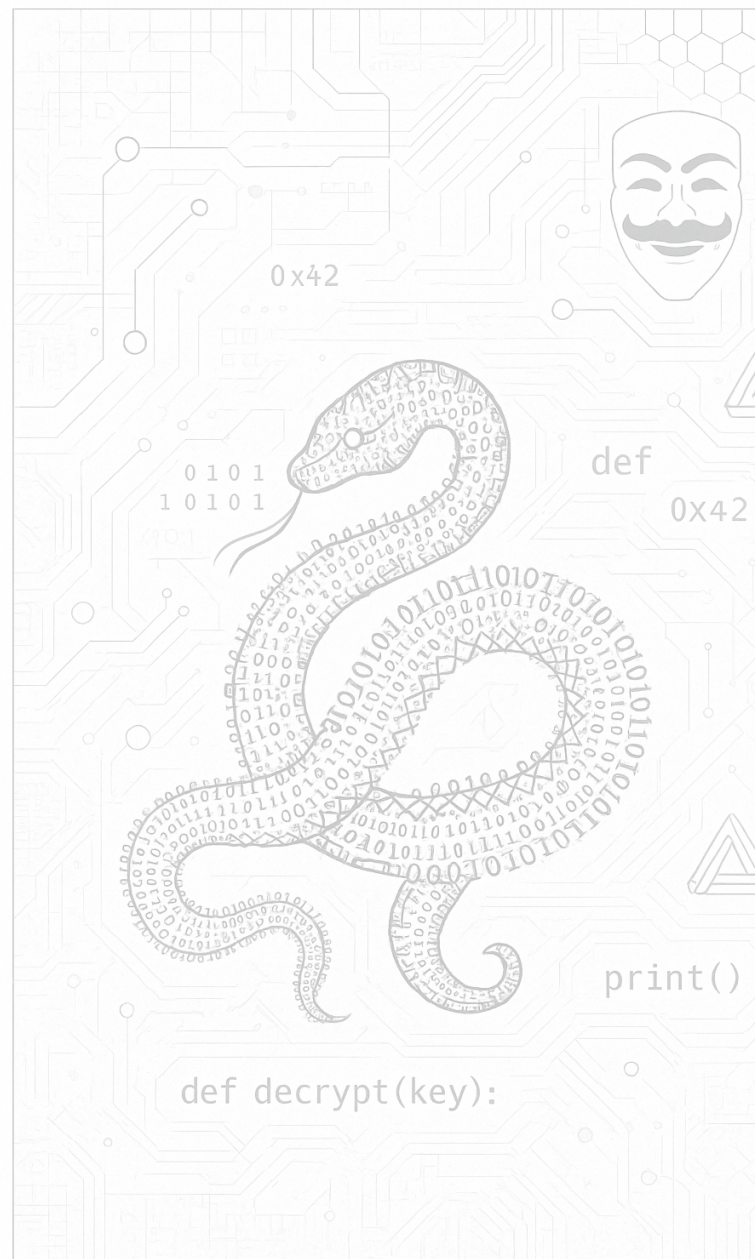
Algorithme : suite finie d'instructions permettant de résoudre un problème.

Complexité : quantité de ressources (temps, mémoire) consommées par un algorithme.

Idiomes Python

```
root@python:~$  
  
# Compréhension de liste  
evens = [x for x in range(10) if x % 2 == 0]  
  
Slicing  
  
s = "NSI"  
print(s[::-1]) # ISN
```

Schéma (image)



Les Chaînes de Caractères en Python

💡 Définition et Création

Une **chaîne de caractères** (ou `string`) est une séquence ordonnée et **immuable** de caractères. On la reconnaît par les guillemets simples `' '` ou doubles `" "` qui l'entourent. Une chaîne de caractères peut contenir des lettres, des chiffres, des symboles et des espaces.

```
root@python:~$  
  
# Création d'une chaîne de caractères vide  
ma_chaine = ""  
# Création d'une chaîne avec du texte  
salutation = "Bonjour le monde"  
# Obtenir la longueur d'une chaîne avec len()  
longueur = len(salutation)  
print(f"La chaîne 'salutation' contient {longueur} caractère")  
# Affiche "La chaîne 'salutation' contient 16 caractères."
```

⚙️ Saisie et Traitement

Saisie avec `input()`

La fonction `input()` permet de demander à l'utilisateur de saisir une chaîne de caractères. Le programme s'arrête en attendant que l'utilisateur entre du texte et appuie sur Entrée. La valeur saisie est renvoyée sous forme de chaîne de caractères.

```
root@python:~$  
  
nom = input("Entrez votre nom : ")  
print(f"Bonjour, {nom} !")  
# Si l'utilisateur tape "Alice", la console affichera : "Bo
```

Méthodes de modification de la casse

Les chaînes de caractères sont **immuables**, ce qui signifie que vous ne pouvez pas modifier un caractère existant directement. Les méthodes suivantes renvoient une **nouvelle** chaîne de caractères modifiée.

`.upper()` : renvoie une nouvelle chaîne avec tous les caractères en majuscules.

`.lower()` : renvoie une nouvelle chaîne avec tous les caractères en minuscules.

`.capitalize()` : renvoie une nouvelle chaîne avec la première lettre en majuscule.

```
root@python:~$  
  
chaine = "Hello World"  
print(chaine.upper()) # Affiche "HELLO WORLD"  
print(chaine.lower()) # Affiche "hello world"
```

😞 Immuabilité

⚠️ Une chaîne de caractères ne peut pas être modifiée après sa création. Toutes les opérations qui semblent "modifier" une chaîne, comme le remplacement d'un caractère, créent en réalité une **nouvelle** chaîne en mémoire.

```
root@python:~$  
  
slogan = "J'adore Python"  
# Cette instruction générera une erreur !  
# slogan[8] = 'C'  
# TypeError: 'str' object does not support item assignment  
# Pour obtenir le résultat souhaité, il faut créer une nouv  
nouveau_slogan = slogan[:8] + "le Python"  
print(nouveau_slogan) # Affiche "J'adore le Python"
```

🔍 Indexation et Accès aux éléments

Chaque caractère d'une chaîne possède une position, appelée **index**. En Python, ⚠️ **l'indexation commence à 0**.

On peut aussi utiliser des index **négatifs** pour accéder aux caractères à partir de la fin de la chaîne. L'index `-1` correspond au dernier caractère, `-2` à l'avant-dernier, et ainsi de suite.

```
root@python:~$  
  
mot = "Python"  
# Indexation positive  
print(mot[0]) # Affiche "P"  
# Indexation négative  
print(mot[-1]) # Affiche "n"  
# Le slicing : Extraire une partie de la chaîne [début:fin]  
# ⚠️ La fin est exclue !  
print(mot[1:4]) # Affiche "yth"  
print(mot[2:]) # Affiche "thon"
```

👉 Parcourir une chaîne

La boucle `for` est l'outil principal pour parcourir une chaîne de caractères.

1. Par caractère :

C'est la méthode la plus simple et la plus courante.

```
root@python:~$  
  
mot = "code"  
for caractere in mot:  
    print(caractere)  
Affiche :  
c  
o  
d  
e
```

2. Par index :

Utile si vous avez besoin de la position du caractère en plus de sa valeur. On utilise la fonction `range(len(chaine))`.

```
root@python:~$  
  
mot = "code"  
for i in range(len(mot)):  
    print(f"Caractère n°{i} : {mot[i]}")  
Affiche :  
Caractère n°0 : c  
Caractère n°1 : o  
Caractère n°2 : d  
Caractère n°3 : e
```

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM

Les Listes en Python

💡 Définition et Création

Une liste est une collection de données ordonnée et modifiable, on parle de **Mutable**. Elle est reconnaissable par les crochets `[]` qui l'entourent. On peut y stocker des éléments de types différents (nombres, chaînes de caractères, booléens, etc.).

root@python:~\$

```
# Création une liste vide
ma_liste = []
# Création une liste avec des éléments
fruits = ["pomme", "banane", "orange"]
# Obtenir la longueur d'une liste avec len()
longueur = len(fruits)
print(f"La liste 'fruits' contient {longueur} éléments.")
# Affiche "La liste 'fruits' contient 3 éléments."
```

⚙️ Modification de la liste

Ajouter des éléments

append(element) : ajoute un élément à la fin de la liste.
insert(index, element) : insère un élément à un index précis.
extend(iterable) : ajoute tous les éléments d'un autre objet (comme une autre liste) à la fin.

root@python:~\$

```
jours = ["lundi", "mardi"]
jours.append("jeudi")
print(jours) # ['lundi', 'mardi', 'jeudi']
jours.insert(2, "mercredi")
print(jours) # ['lundi', 'mardi', 'mercredi', 'jeudi']
week_end = ["vendredi", "samedi"]
jours.extend(week_end)
print(jours)
# ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi']
```

Supprimer des éléments

pop(index) : supprime l'élément à l'index donné et le renvoie. Si l'index est omis, il supprime et renvoie le dernier élément.

root@python:~\$

```
nombres = [10, 20, 30, 40]
# Supprimer le dernier élément et le stocker
element_supprime = nombres.pop()
print(f"Élément supprimé : {element_supprime}")
# Élément supprimé : 40
print(nombres) # [10, 30]
```

😞 Copier une liste

⚠️ Si vous attribuez une liste à une autre variable avec le signe `=`, vous ne faites pas une copie ! Les deux variables pointent vers le même objet en mémoire. On appelle cela une référence.

Pour faire une vraie copie, on peut utiliser la technique du slicing ou la méthode `.copy()`.

root@python:~\$

```
liste_a = [1, 2, 3]
liste_b = liste_a # C'est une référence, pas une copie !
liste_b.append(4)
print(liste_a) # Affiche [1, 2, 3, 4] !
liste_c = liste_a[:] # Vraie copie avec slicing
liste_c.append(5)
print(liste_a) # Affiche [1, 2, 3, 4]
print(liste_c) # Affiche [1, 2, 3, 4, 5]
```

🔍 Indexation et Accès aux éléments

Chaque élément d'une liste possède une position, appelée index. En Python, ⚠️ l'indexation commence à 0.

On peut aussi utiliser des index négatifs pour accéder aux éléments à partir de la fin de la liste. L'index -1 correspond au dernier élément, -2 à l'avant-dernier, et ainsi de suite.

root@python:~\$

```
animaux = ["chien", "chat", "oiseau", "poisson"]
# Indexation positive
print(animaux[0]) # Affiche "chien"
# Indexation négative
print(animaux[-1]) # Affiche "poisson"
# Le slicing : Extraire une partie de la liste [début:fin]
# ⚠️ La fin est exclue !
print(animaux[1:3]) # Affiche ['chat', 'oiseau']
print(animaux[2:]) # Affiche ['oiseau', 'poisson']
```

👁️ Parcourir une liste

La boucle for est l'outil principal pour parcourir une liste.

1. Par élément :

C'est la méthode la plus simple et la plus courante. On parcourt directement chaque valeur de la liste.

root@python:~\$

```
pizzas = ["reine", "calzone"]
for pizza in pizzas:
    print(pizza)
```

Affiche :
reine
calzone

2. Par index :

Utilise si vous avez besoin de la position de l'élément en plus de sa valeur. On utilise la fonction `range(len(liste))`.

root@python:~\$

```
pizzas = ["reine", "calzone"]
for i in range(len(pizzas)):
    print(f"Pizza n°{i} : {pizzas[i]}")
```

Affiche :
Pizza n°0 : reine
Pizza n°1 : calzone

3. Parcours mixte (avec enumerate) :

La méthode la plus élégante pour avoir à la fois l'index et l'élément. Elle est à privilégier par rapport à la méthode précédente.

root@python:~\$

```
pizzas = ["reine", "calzone"]
for index, pizza in enumerate(pizzas):
    print(f"Index {index} correspond à la {pizza}.")
```

Affiche :
Index 0 correspond à la reine.
Index 1 correspond à la calzone.

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM

Les Structures de Données

Les Listes (list)

- Collection ordonnée et modifiable d'éléments.
- Permet les doublons.

```
root@python:~$
```

```
ma_liste = [1, "a", 3.14, 1]
ma_liste.append(5)
print(ma_liste[1]) # a
```

Les Dictionnaires (dict)

Collection non-ordonnée de paires clé-valeur. Mutable.

```
root@python:~$
```

```
personne = {"nom": "Alan", "age": 41}
print(personne["nom"]) # Alan
```

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM